

MECHATRONICS AND IOT

ASSIGNMENT - 4

Smart Refrigerator Monitoring System

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

Smart Refrigerator Monitoring System

1. Project Overview

The Smart Refrigerator Monitoring System is an IoT-based system designed to continuously monitor the internal temperature and humidity of a refrigerator. Maintaining suitable temperature and humidity conditions is important for preserving food quality, preventing spoilage, and improving energy efficiency.

The system uses a **DHT11 temperature and humidity sensor** to measure the environmental conditions inside the refrigerator. An **ESP8266 NodeMCU** collects the sensor data and transmits it through Wi-Fi to a cloud platform. The stored data can be monitored remotely and analyzed to identify abnormal temperature or humidity conditions.

Main components

- ESP8266 Node MCU
- DHT11/DHT22 Sensor
- OLED/LCD Display
- Buzzer
- Relay Module
- Breadboard
- Jumper Wires

2. Project Statement

Food stored in refrigerators requires suitable temperature and humidity conditions for proper preservation. Manual checking of refrigerator conditions does not provide continuous monitoring and may fail to detect sudden changes.

Therefore, an IoT-based Smart Refrigerator Monitoring System is proposed to continuously measure internal temperature and humidity and provide remote access to the sensor data through a cloud platform.

3. Proposed Solution

The ESP8266 processes the sensor readings and sends them to the cloud using Wi-Fi. The user can view the readings remotely through a cloud dashboard.

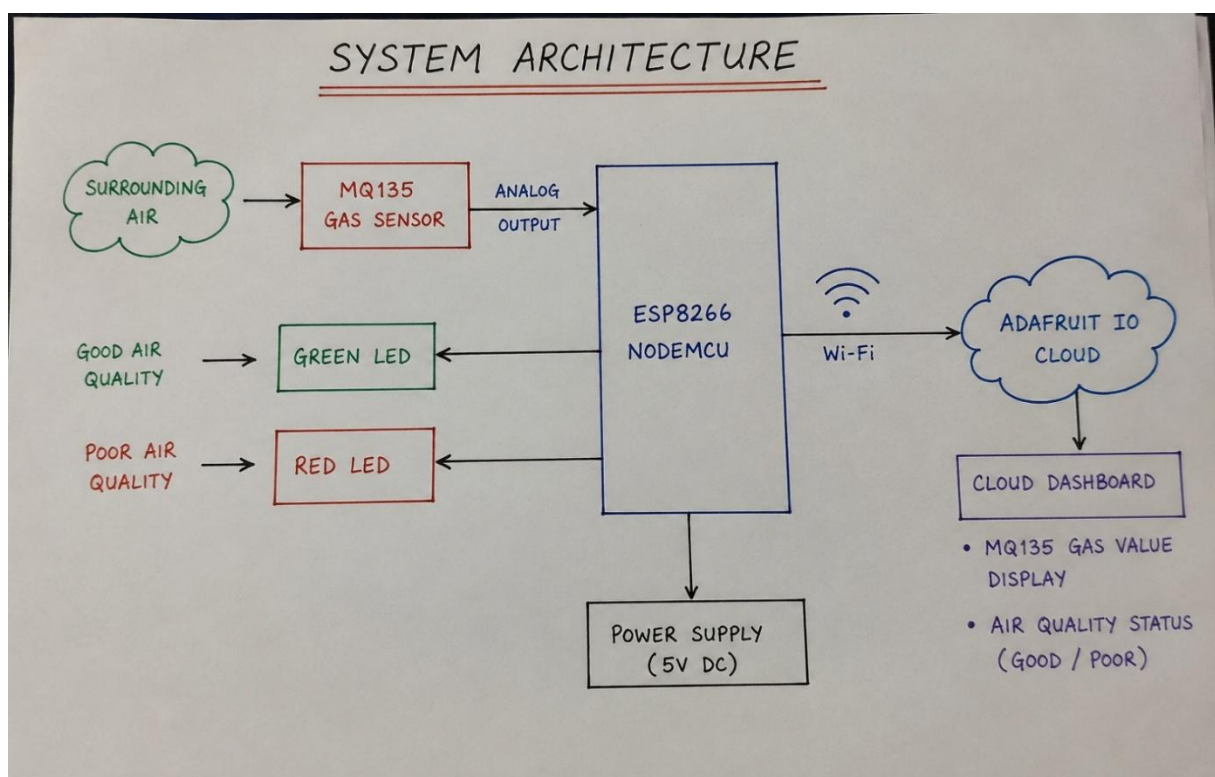
The system can also identify abnormal conditions, such as temperature becoming too high or too low, which can indicate a possible refrigeration problem.

The system operates as follows:

1. **Temperature Monitoring:** Continuously measures the internal refrigerator temperature using the DHT11 sensor. ESP8266 reads the sensor value.
2. **Humidity Monitoring:** Measures the relative humidity inside the refrigerator. If the value is below the threshold, the air is considered GOOD.
3. **Data Processing:** The ESP8266 receives and processes the sensor readings. If the value exceeds the threshold, the air is considered POOR/POLLUTED.

4. **Wireless Communication:** The ESP8266 uses Wi-Fi to transmit the collected data to the cloud. The MQ135 value and air-quality status are transmitted to Adafruit IO using Wi-Fi.
5. **Cloud Data Storage:** Temperature and humidity readings are stored on the selected cloud platform.

4. System Architecture:



5. Circuit Table:

CIRCUIT TABLE

1. ESP8266 NODEMCU CONNECTION

Component	Pin	Description / Connection	Function
 ESP8266 NodeMCU	VIN	5V DC (USB / External 5V)	Power Supply (5V)
	GND	GND	Ground
	3V3	3.3V Output	3.3V Power Output
	D1 (GPIO5)	SCL (for OLED/I2C)	I2C Clock
	D2 (GPIO4)	SDA (for OLED/I2C)	I2C Data
	D5 (GPIO14)	DHT Data Pin	Connect to DHT11/DHT22 Data
	D6 (GPIO12)	Buzzer Positive (+)	Buzzer Control
	D7 (GPIO13)	Relay IN	Relay Control Signal

2. DHT11 / DHT22 SENSOR CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
 DHT11 / DHT22 Sensor	VCC	3.3V	Power Supply (3.3V)
	DATA	D5 (GPIO14)	Data Signal
	GND	GND	Ground

3. OLED / LCD DISPLAY CONNECTION (I2C)

Component	Pin	ESP8266 (NodeMCU) Pin	Function
 OLED Display (I2C)	VCC	3.3V	Power Supply (3.3V)
	GND	GND	Ground
	SCL	D1 (GPIO5)	I2C Clock
	SDA	D2 (GPIO4)	I2C Data

4. BUZZER CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
 Buzzer	Positive (+)	D6 (GPIO12)	Buzzer Control (HIGH/LOW)
	Negative (-)	GND	Ground

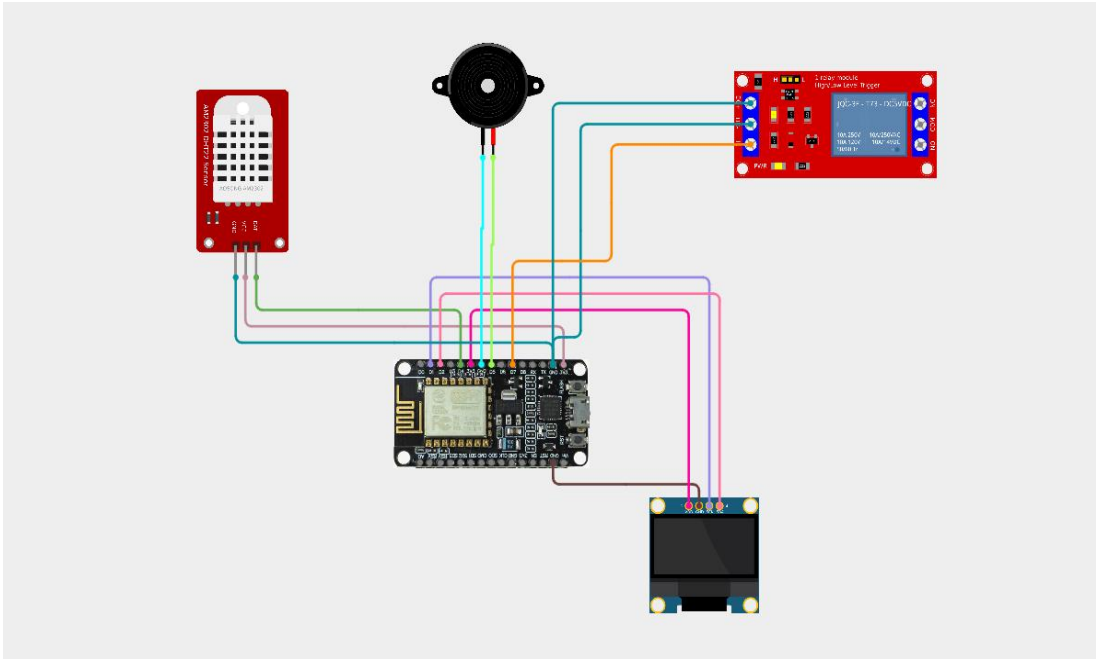
5. RELAY MODULE CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
 Relay Module	VCC	5V	Power Supply (5V)
	GND	GND	Ground
	IN	D7 (GPIO13)	Relay Control Signal

6. OTHER COMPONENTS

Component	Description	Function
Breadboard	Solderless breadboard for making connections	Used to build and test the circuit
Jumper Wires	Male to Male / Female to Male jumper wires	Used for connections between components

6. Circuit Design:



7. Algorithm:

Step 1: Start the system.

Step 2: Initialize ESP8266 and DHT11 sensor.

Step 3: Connect ESP8266 to the Wi-Fi network.

Step 4: Establish connection with the cloud platform.

Step 5: Initialize the DHT11 sensor for temperature and humidity measurement.

Step 6: Read the temperature value from the DHT11 sensor.

Step 7: Read the humidity value from the DHT11 sensor.

Step 8: Check whether the sensor readings are valid.

- If the readings are invalid, repeat the sensor reading.
- If the readings are valid, continue to the next step.

Step 9: Compare the measured temperature with the predefined safe temperature range.

- If the temperature is within the safe range, set the refrigerator condition as normal.
- If the temperature exceeds the safe range, identify the condition as high temperature.

Step 10: Compare the measured humidity with the predefined humidity range.

- If the humidity is within the desired range, set humidity status as normal.
- If the humidity is outside the desired range, identify it as an abnormal humidity condition.

8. Coding:

```
#include <WiFi.h>

#include <Wire.h>

#include <Adafruit_MQTT.h>

#include <Adafruit_MQTT_Client.h>

#include <DHT.h>

#include <hd44780.h>

#include <hd44780ioClass/hd44780_I2Cexp.h>

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"

#define WIFI_PASSWORD "qkts5737"

// =====

// ADAFRUIT IO DETAILS

// =====

#define AIO_SERVER    "io.adafruit.com"

#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "Sivasaravanan"

#define AIO_KEY       "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"

// =====

// DHT11

// =====

#define DHTPIN 15

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// =====

// I2C LCD

// =====
```

```
hd44780_I2Cexp lcd;

#define SDA_PIN 21
#define SCL_PIN 22

// =====
// RELAY
// =====

#define RELAY_PIN 26

// Most common relay modules are ACTIVE LOW

#define RELAY_ON LOW

#define RELAY_OFF HIGH

// =====
// BUZZER
// =====

#define BUZZER_PIN 25

// =====
// MQTT
// =====

WiFiClient client;

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// =====
// ADAFRUIT IO FEEDS
// =====

// Temperature feed
```

```
Adafruit_MQTT_Publish temperatureFeed =  
  Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/hvac-temperature"  
  );  
// Humidity feed  
Adafruit_MQTT_Publish humidityFeed =  
  Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/hvac-humidity"  
  );  
// Status feed  
Adafruit_MQTT_Publish statusFeed =  
  Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/hvac-status"  
  );  
// Buzzer feed  
Adafruit_MQTT_Publish buzzerFeed =  
  Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/hvac-buzzer"  
  );  
// =====  
// CONNECT TO WIFI  
// =====  
void connectWiFi()  
{  
  Serial.println();
```

```
Serial.println("Connecting to WiFi...");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}

Serial.println();

Serial.println("WiFi Connected!");

Serial.print("IP Address: ");

Serial.println(WiFi.localIP());
}

// =====
// CONNECT TO ADAFRUIT IO
// =====

bool connectMQTT()
{
    if (mqtt.connected())
    {
        return true;
    }

    Serial.println();

    Serial.println("Connecting to Adafruit IO...");

    int8_t ret = mqtt.connect();

    if (ret != 0)
    {
        Serial.print("MQTT Connection Failed: ");
```

```

    Serial.println(mqtt.connectErrorString(ret));

    mqtt.disconnect();

    return false;
}

Serial.println("Adafruit IO Connected!");

return true;
}

// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);

    // -----

    // DHT11
    // -----

    dht.begin();

    // -----

    // I2C LCD
    // -----

    Wire.begin(SDA_PIN, SCL_PIN);

    int lcdStatus = lcd.begin(16, 2);

    lcd.backlight();

    lcd.clear();

    lcd.setCursor(0, 0);

    lcd.print("HVAC MONITOR");

    lcd.setCursor(0, 1);

    lcd.print("Starting...");

    // -----

```

```
// RELAY
// -----
pinMode(RELAY_PIN, OUTPUT);
digitalWrite(RELAY_PIN, RELAY_OFF);
// -----
// BUZZER
// -----
pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(BUZZER_PIN, LOW);
delay(2000);
// -----
// WIFI
// -----
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("WiFi Connecting");
connectWiFi();
// -----
// MQTT
// -----
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Adafruit IO");
lcd.setCursor(0, 1);
lcd.print("Connecting...");
connectMQTT();
delay(2000);
lcd.clear();
}
```

```
// =====  
// LOOP  
// =====  
void loop()  
{  
  // =====  
  // CHECK WIFI  
  // =====  
  if (WiFi.status() != WL_CONNECTED)  
  {  
    Serial.println("WiFi disconnected!");  
    connectWiFi();  
  }  
  // =====  
  // CHECK MQTT  
  // =====  
  if (!connectMQTT())  
  {  
    Serial.println("MQTT not connected.");  
    delay(5000);  
    return;  
  }  
  // =====  
  // READ DHT11  
  // =====  
  float temperature = dht.readTemperature();  
  float humidity = dht.readHumidity();  
  // =====  
  // SENSOR ERROR
```

```

// =====
if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 ERROR!");
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("DHT11 ERROR");
    lcd.setCursor(0, 1);
    lcd.print("Check Wiring");
    digitalWrite(RELAY_PIN, RELAY_OFF);
    digitalWrite(BUZZER_PIN, LOW);
    delay(2000);
    return;
}

// =====

// HVAC STATUS

// =====

String hvacStatus;

String buzzerStatus;

if (temperature > 30.0)
{
    // High temperature
    digitalWrite(RELAY_PIN, RELAY_ON);
    digitalWrite(BUZZER_PIN, HIGH);
    hvacStatus = "HIGH TEMPERATURE";
    buzzerStatus = "ON";
}

else
{

```

```

// Normal temperature
digitalWrite(RELAY_PIN, RELAY_OFF);
digitalWrite(BUZZER_PIN, LOW);
hvacStatus = "NORMAL";
buzzerStatus = "OFF";
}
// =====

// LCD DISPLAY
// =====

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Temp:");
lcd.print(temperature, 1);
lcd.write(223);
lcd.print("C");
lcd.setCursor(0, 1);
lcd.print("Hum:");
lcd.print(humidity, 1);
lcd.print("%");
// =====

// SERIAL MONITOR
// =====

Serial.println();
Serial.println("=====");
Serial.print("Temperature : ");
Serial.print(temperature, 1);
Serial.println(" C");
Serial.print("Humidity  : ");
Serial.print(humidity, 1);

```

```
Serial.println(" %");
Serial.print("HVAC Status : ");
Serial.println(hvacStatus);
Serial.print("Relay    : ");
if (temperature > 30.0)
    Serial.println("ON");
else
    Serial.println("OFF");
Serial.print("Buzzer    : ");
Serial.println(buzzerStatus);

// =====
// SEND TEMPERATURE
// =====

if (temperatureFeed.publish(temperature))
{
    Serial.println("Temperature -> Adafruit IO : OK");
}
else
{
    Serial.println("Temperature -> Adafruit IO : FAILED");
}

// =====
// SEND HUMIDITY
// =====

if (humidityFeed.publish(humidity))
{
    Serial.println("Humidity -> Adafruit IO : OK");
}
else
```

```
{
  Serial.println("Humidity -> Adafruit IO : FAILED");
}

// =====

// SEND HVAC STATUS

// =====

if (statusFeed.publish(hvacStatus.c_str()))
{
  Serial.println("Status -> Adafruit IO : OK");
}
else
{
  Serial.println("Status -> Adafruit IO : FAILED");
}

// =====

// SEND BUZZER STATUS

// =====

if (buzzerFeed.publish(buzzerStatus.c_str()))
{
  Serial.println("Buzzer -> Adafruit IO : OK");
}
else
{
  Serial.println("Buzzer -> Adafruit IO : FAILED");
}

Serial.println("=====");

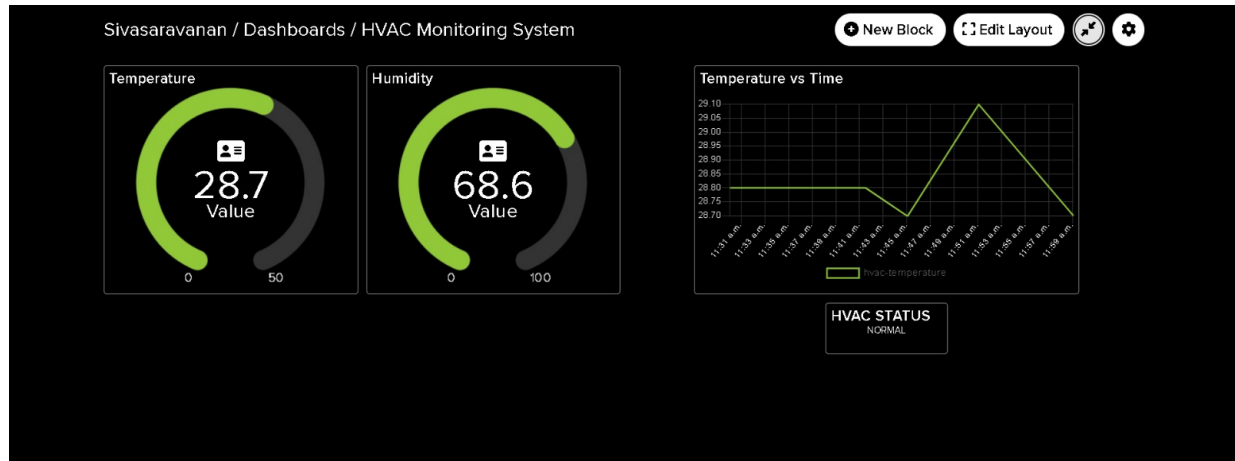
// =====

// WAIT 5 SECONDS

// =====
```

```
delay(5000);  
}
```

9. System Working :



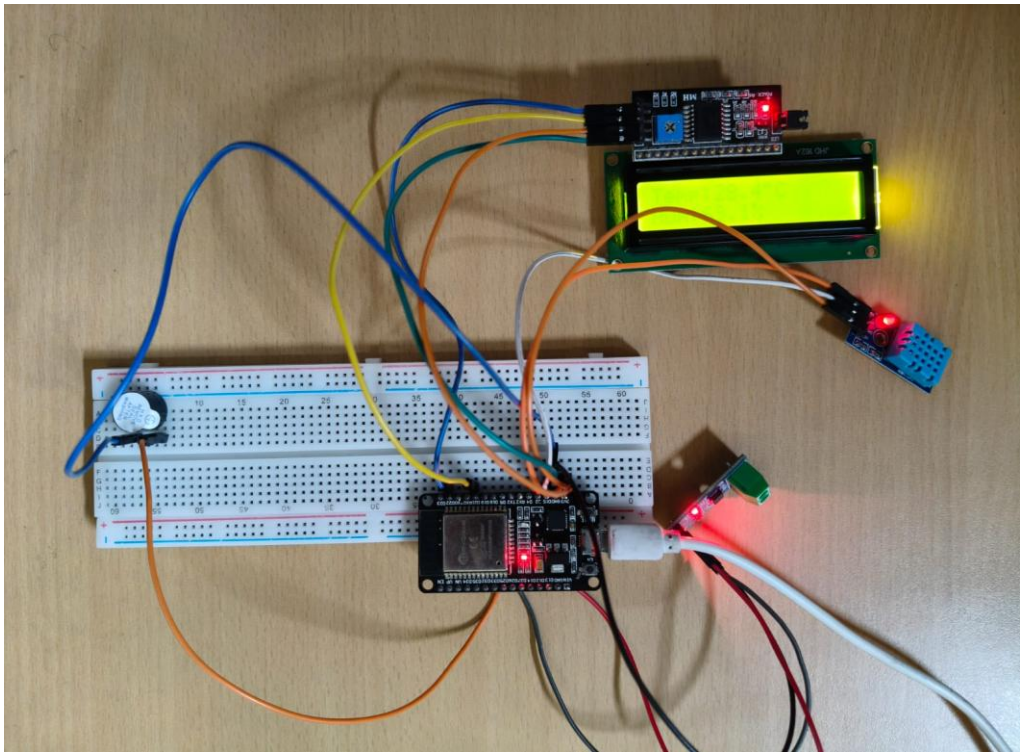
- sensor and connects to the Wi-Fi network.
- The DHT11 continuously measures the temperature and humidity inside the refrigerator. The ESP8266 reads these values at regular intervals.
- The ESP8266 sends these readings to the cloud platform. The user can access the dashboard remotely and observe the refrigerator conditions.
- If the temperature rises significantly above the desired range, it can indicate that the refrigerator door has been left open, the cooling system is not functioning properly, or there is another refrigeration issue.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component		Specification / Description	Quantity
1.		ESP8266 NodeMCU	ESP8266 Wi-Fi Development Board	1
2.		DHT11 Temperature & Humidity Sensor	Digital Temperature & Humidity Sensor (3.3V – 5V)	1
3.		Breadboard	Solderless Breadboard	1
4.		Jumper Wires	Male to Male / Female to Male Jumper Wires	As required
5.		USB Cable	Micro USB Cable for Power and Programming	1
6.		Computer/Laptop	Laptop or Computer for Programming and Monitoring	1
7.		Wi-Fi Connection	Internet Connection for ESP8266	1
8.		Cloud Account	Adafruit IO Cloud Platform for Data Monitoring	1

11. Prototype Implementation:



- The prototype is implemented using an ESP8266 NodeMCU and DHT11 sensor. The DHT11 is positioned to represent the internal environment of a refrigerator.
- The sensor is connected to the ESP8266 through the DATA pin. The ESP8266 reads the temperature and humidity values and uses its Wi-Fi capability to transmit the information to the cloud.
- A cloud dashboard can display the data in the form of gauges, numerical values, and graphs. This allows the refrigerator's internal environmental conditions to be monitored remotely.

12. Results and Performance Analysis:

The developed Smart Refrigerator Monitoring System was successfully designed and implemented using a DHT11 temperature and humidity sensor, ESP8266 NodeMCU, Wi-Fi connectivity, and a cloud monitoring platform. The main objective of the system was to continuously monitor the environmental conditions inside a refrigerator and make the measured data available for remote monitoring.

The DHT11 sensor successfully measured the internal temperature and relative humidity. The ESP8266 received the sensor readings through the digital data pin and processed the values before transmitting them to the cloud platform.

The ESP8266 successfully established a Wi-Fi connection and transmitted the temperature and humidity readings to the cloud. The cloud dashboard can display the current values as numerical readings, gauges, or graphs.

Temperature Monitoring

The temperature monitoring function continuously observes the internal refrigerator temperature. A predefined temperature range can be used to determine whether the refrigerator is operating under normal conditions.

Humidity Monitoring

The DHT11 also measures the relative humidity inside the refrigerator. Monitoring humidity is useful because excessive moisture can affect food quality and may contribute to condensation.

The humidity value is transmitted along with the temperature reading and can be observed on the cloud dashboard. Historical humidity data can also be analyzed to identify unusual changes over time.

Cloud Monitoring Performance

The ESP8266 uses its built-in Wi-Fi capability to transmit the sensor readings to the cloud platform. The cloud system provides remote access to the collected data.

The following operations were successfully demonstrated:

- DHT11 temperature measurement.
- DHT11 humidity measurement.
- ESP8266 sensor-data processing.
- Wi-Fi connectivity.
- Cloud data transmission.
- Remote dashboard monitoring.
- Continuous sensor-data updates.
- Historical data storage and visualization.

Advantages of the System

The proposed system provides several advantages over manual refrigerator monitoring:

1. Continuous Monitoring: Temperature and humidity are monitored at regular intervals.

2. Remote Access: Users can monitor refrigerator conditions remotely through the cloud dashboard.
3. Early Problem Detection: A sudden increase in temperature can indicate a possible cooling or door-opening problem.
4. Data Storage: Sensor readings can be stored in the cloud for future analysis.
5. Trend Analysis: Historical graphs can help identify temperature and humidity fluctuations.

Overall Performance

The prototype successfully demonstrates the concept of an IoT-based Smart Refrigerator Monitoring System. The DHT11 sensor provides temperature and humidity information, while the ESP8266 acts as the processing and wireless communication unit. The cloud platform enables remote monitoring and storage of the collected data.

The system can be further improved by adding a buzzer for abnormal-temperature alerts, an OLED/LCD display for local monitoring, a door-open sensor, and automatic control of the cooling system.